Chapter 2

陣列結構

2.1 前言

- 「陣列」(Array) 結構具有以下重要特徵：
  - 一個陣列可以包含許多個陣列元素，陣列元素之個數又稱為陣列之大小。
  - 同一個陣列裡的元素都具備相同的資料型態 (Data Type)。
  - 為方便資料的存取，可將陣列設計成一維 (Dimension)、二維、三維，...，甚至更多維的陣列。

2.1 前言

- 宣告一個陣列時，作業系統會在記憶體空間指定一個起始位置給該陣列，並且根據陣列的資料型態和大小來配予一組連續的記憶體位置。
- 陣列元素在記憶體中的位置是連續且相鄰的，因此只要透過一套簡單的陣列元素位址計算公式，便能得知某陣列元素所在之記憶體位置。
- 經由陣列索引 (Index) ，可以迅速且直接地存取陣列中的元素資料。

2.2 宣告陣列

- 宣告陣列時須定義下列幾項屬性：
  - 陣列的資料型態。
  - 陣列的名稱。
  - 陣列的維度，即中括號 ([ ]) 的個數。
  - 陣列每一維度的元素個數，亦即陣列大小。
  - 陣列元素的初值，此項可以省略。

2.2 宣告陣列

- 以 C 語言為例，宣告陣列的語法如下：

【陣列宣告語法】

1.資料型態 陣列名稱[陣列大小][陣列大小] ・・・[陣列大小];

2.資料型態 陣列名稱[陣列大小][陣列大小]・・・[陣列大小] = {初值};

2.2.1 一維陣列

【例 1 】宣告一個可以用來存放 6 次考試成績的整數陣列。

```
int grades[6] = {83, 75, 92, 74, 88, 93};
```

grades[0]	grades[1]	grades[2]	grades[3]	grades4]	grades[5]
83	75	92	74	88	93

圖 2.1 整數型態的一維陣列 grades[6]

2.2.2 二維陣列

【例 2 】宣告二維陣列。

```
int matrix[3][4] = { {1,2,3,4},      /* 設定第 0 列的初始值 */
                     {5,6,7,8},      /* 設定第 1 列的初始值 */
                     {9,10,11,12}    /* 設定第 2 列的初始值 */
                   };
```

matrix	[0]	[1]	[2]	[3]
[0]	1	2	3	4
[1]	5	6	7	8
[2]	9	10	11	12

圖 2.2 整數型態的二維陣列 matrix[3][4]

2.2.2 二維陣列

【例 3 】宣告 grades[50][6] 為一個可以用來儲存 50 位同學之 5 次資料結構考試成績之二維陣列。

```
int grades[50][6] ;
```

	學號	成績 1	成績 2	成績 3	成績 4	成績 5
grades	[0]	[1]	[2]	[3]	[4]	[5]
[0]	920001	88	63	78	85	95
[1]	920002	76	78	80	90	92
[2]	920003	87	66	91	84	80
...	...					
[49]	920050	91	85	78	69	90

圖 2.3 整數型態的二維陣列 grades[50][6]

2.3 陣列的表示法

- 一維陣列的表示法
- 二維陣列的表示法
- 三維陣列的表示法
- 多維度陣列的表示法
- 上三角矩陣
- 下三角矩陣
- 我們將「陣列」與「矩陣」視為同義詞

2.3.1 一維陣列的表示法

- A[n] = {A[0] ， A[1] ，… ， A[n-1]}
- 假設
  - 一維陣列 A 在記憶體的起始位置為 α 。
  - 每個陣列元素所需的儲存空間是 z 個位元組 (Byte) 。
  - 陣列元素 A[i] 的記憶體位址為 Loc(A[i]) 。
- 則：

  Loc(A[i]) = A 的起始位址 + A[i] 相對於陣列起始位置之位移

  = α + i × z 。

2.3.1 一維陣列的表示法

【例 1 】設陣列 A 是一個大小為 10 的一維陣列，且陣列 A 在記憶體之起始位置為 1000 ，且每個元素都需要 2 個位元組的儲存空間。則 A[7] 的記憶體位置為何？

【解答】

Loc(A[7]) = A 的起始位址 + A[7] 相對於陣列起始位置之位移

= 1000 + 7 × 2

= 1014 。

2.3.1 一維陣列的表示法

【例 2 】設 d[100] 為一個實數陣列，每一個元素佔 4 個位元組。若 d[10] 的位址為 2000 ，則 d[88] 的位址為何？

【解答】

Loc(d[88]) = d[10] 的位址 + d[88] 相對於 d[10] 之位移

= 2000 + (88-10) × 4

= 2312 。

2.3.1 一維陣列的表示法

- A[f:t] = {A[f] ， A[f-1] ， … ， A[0] ，
  A[1] ， … ， A[t] } 。
- 假設
  - 一維陣列 A[f] 在記憶體的起始位置為 α 。
  - 每個陣列元素所需的儲存空間是 z 個位元組 (Byte) 。
  - 陣列元素 A[i] 的記憶體位址為 Loc(A[i]) 。
- 則：

  Loc(A[i]) = A 的起始位址 + A[i] 相對於陣列起始位置之位移
  = α + (i-f) × z 。

2.3.1 一維陣列的表示法

【例 3 】設 A[-15:25] 在記憶體的起始位置為 123 ，且每一元素佔 4 個位元組，則 A[3] 的位址為何？

【解答】

Loc(A[3]) = A[-15:25] 的起始位址 + A[3] 相對於 A[-15:25] 之位移

= 123 + (3-(-15)) × 4

= 195 。

2.3.2 二維陣列的表示法

- B[m][n] = {B[0][0] ， B[0][1] ，… ，B[m-1][n-1] } 。
- 共計有 m×n 個元素。

B	[0]	[1]	...	[n-1]
[0]	[0][0]	[0][1]	...	[0][n-1]
[1]	[1][0]	[1][1]	...	[1][n-1]
...	...	...	...	...
[m-1]	[m-1][0]	[m-1][1]	...	[m-1][n-1]

圖 2.4 二維陣列 B[m][n]

2.3.2 二維陣列的表示法

- 將二維的陣列元素轉換成一維排列的方式有下列兩種：
  - 以列為主 (Row Major) 。
  - 以行為主 (Column Major) 。

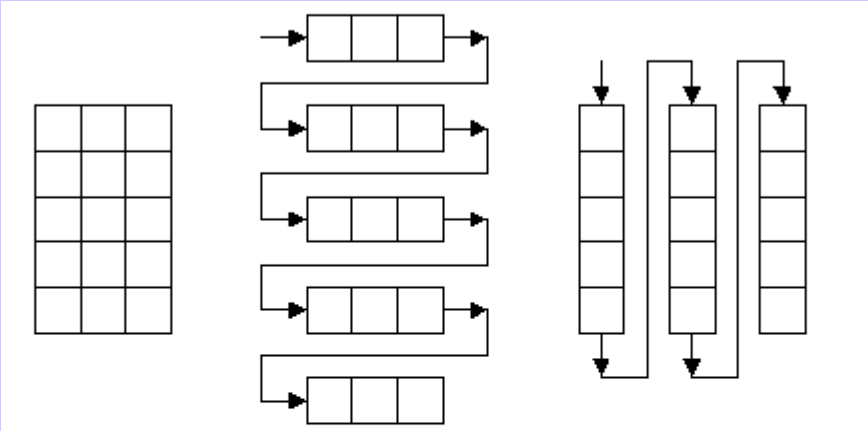


圖 2.5 二維陣列、以列爲主、以行爲主之排列順序

2.3.2 二維陣列的表示法

- 以列為主 (Row Major) 的表示法：
- 假設
  - 假設二維陣列 B 在記憶體的起始位置為 α 。
  - 陣列之個別元素所需的記憶體空間大小為 z 。
  - 陣列元素 B[i][j] 的記憶體位址為 Loc(B[i][j]) 。
- 則：

$Loc_{RM}(B[i][j])$ = B 的起始位址 + B[i][j] 相對於陣列起始位置之位移

$= \alpha + (i \times n + j) \times z$ 。

2.3.2 二維陣列的表示法

- 以行為主 (Column Major) 的表示法：
- 假設
  - 假設二維陣列 B 在記憶體的起始位置為 α 。
  - 陣列之個別元素所需的記憶體空間大小為 z 。
  - 陣列元素 B[i][j] 的記憶體位址為 Loc(B[i][j]) 。
- 則：

$Loc_{CM}(B[i][j])$ = B 的起始位址 + B[i][j] 相對於陣列起始位置之位移

$= \alpha + (j \times m + i) \times z$ 。

2.3.2 二維陣列的表示法

【例 1 】已知 A[1][2] 之記憶體位置為 22 ，且 A[2][4] 和 A[4][2] 之記憶體位置分別為 30 和 40 ，則 A[5][5] 之位置為何？

【解答】

因為 A[2][4] 的記憶體位置 30 小於 A[4][2] 的記憶體位置 40 ，可見該陣列是採「以列為主」排列。因此：

$Loc_{RM}(A[i][j])$ = A 的起始位址 + A[i][j] 相對於陣列起始位置之位移

$= \alpha + (i \times n + j) \times z$ 。

2.3.2 二維陣列的表示法

Loc_{RM}(A[2][4]) = A[1][2] 的位址 + A[2][4] 相對於 A[1][2] 之位移

= 22 + ((2-1) × n + (4-2)) × z

= 22 + (n + 2) × z = 30 。

所以， nz + 2z = 8 ……(1)

Loc_{RM}(A[4][2]) = A[1][2] 的位址 + A[4][2] 相對於 A[1][2] 之位移

= 22 + ((4-1) × n + (2-2)) × z

= 22 + (3n + 0) × z = 40 。

所以， 3nz = 18 ， nz = 6 ……(2) ，將 (2) 代入 (1) 得到

6 + 2z = 8 ， z = 1 ，代入 (2) 得到 n = 6 ，因此

2.3.2 二維陣列的表示法

LocRM(A[5][5]) = A[1][2] 的位址 + A[5][5] 相對於 A[1][2] 之位移

= 22 + ((5-1) × n + (5-2)) × z

= 22 + (4 × 6 + 3) × 1

= 49 。

2.3.2 二維陣列的表示法

【例 2 】設 A[f1:t1][f2:t2] 為一個二維陣列宣告，其記憶體起始位置為 α ，每個元素所佔的記憶體空間為 z 。則陣列元素 A[i][j] 之位置為何？

【解答】

- 已知 A[f1][f2] 在記憶體之起始位置為 α ，且每個元素所佔的記憶體空間為 z 。
- 設二維陣列 A 共有 m 列 n 行，則
  m = t1-f1+1 ，
  n = t2-f2+1 。

2.3.2 二維陣列的表示法

- 以列為主

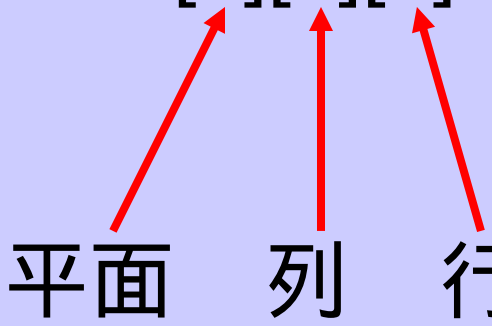
$$Loc_{RM}(A[i][j]) = A[f1][f2] \text{ 的起始位址 } + A[i][j] \text{ 相對於 } A[f1][f2] \text{ 之位移}$$
$$= \alpha + ((i-f1) \times n + (j-f2)) \times z$$

- 以行為主

$$Loc_{CM}(A[i][j]) = A[f1][f2] \text{ 的起始位址 } + A[i][j] \text{ 相對於 } A[f1][f2] \text{ 之位移}$$
$$= \alpha + ((j-f2) \times m + (i-f1)) \times z$$

2.3.3 三維陣列的表示法

```
int cubic[3][2][3] = { {1,2,3,4,5,6},
                       {7,8,9,1,2,3},
                       {4,5,6,7,8,9}};
```



- 它包括 3 個平面，每個平面有 2 列及 3 行。

2.3.3 三維陣列的表示法

- 假設我們宣告了一個三維陣列 C[l][m][n]
  - l 代表第一個維度的大小。
  - m 代表第二個維度的大小。
  - n 代表第三個維度的大小。
  - 共計有 l×m×n 個陣列元素。
- 假設
  - C[0][0][0] 在記憶體中的起始位置為 α 。
  - 陣列之個別元素所需的記憶體空間大小為 z 。

2.3.3 三維陣列的表示法

- 以列為主 (Row Major) 的表示法：
  - 共計有 l 個平面，每個平面有 m 列與 n 行。

$$Loc_{RM}(C[i][j][k]) = \alpha + (i \times m \times n + j \times n + k) \times z$$

- 以行為主 (Column Major) 的表示法：
  - 共計有 n 個平面，每個平面有 l 列與 m 行。

$$Loc_{CM}(C[i][j][k]) = \alpha + (k \times l \times m + j \times l + i) \times z$$

2.3.4 多維度陣列的表示法

- 假設
  - n 維度陣列 $D[s_1][s_2][s_3]\ldots[s_n]$ 在記憶體中的起始位置為 α。
  - 其中 s_1，s_2，…，s_n 分別為 D 陣列的第 1 維、第 2 維，…，第 n 維的元素個數。
  - 每個陣列元素所須要的記憶體儲存空間為 z。

2.3.4 多維度陣列的表示法

- 以列為主 (Row Major) 的表示法：

$$Loc_{RM}(D[i_1][i_2]\ldots[i_n]) = \alpha$$
$$+(i_1 \times s_2 \times s_3 \times \ldots \times s_{n-1} \times s_n$$
$$+ i_2 \times s_3 \times s_4 \times \ldots \times s_{n-1} \times s_n$$
$$+ i_3 \times s_4 \times s_5 \times \ldots \times s_{n-1} \times s_n$$
$$+ \ldots$$
$$+ i_{n-3} \times s_{n-2} \times s_{n-1} \times s_n$$
$$+ i_{n-2} \times s_{n-1} \times s_n$$
$$+ i_{n-1} \times s_n$$
$$+ i_n) \times z \text{ 。}$$

2.3.4 多維度陣列的表示法

- 以行為主 (Column Major) 的表示法：

$$Loc_{CM}(D[i_1][i_2]\ldots[i_n]) = \alpha$$
$$+(i_n \times s_1 \times s_2 \times \ldots \times s_{n-2} \times s_{n-1}$$
$$+ i_{n-1} \times s_1 \times s_2 \times \ldots \times s_{n-3} \times s_{n-2}$$
$$+ i_{n-2} \times s_1 \times s_2 \times \ldots \times s_{n-4} \times s_{n-3}$$
$$+ \ldots$$
$$+ i_3 \times s_1 \times s_2$$
$$+ i_2 \times s_1$$
$$+ i_1) \times z \text{ 。}$$

2.3.5 對角線矩陣的表示法

- 對角線矩陣 (Diagonal Matrix)
  - 是一個「方陣」。
  - 除了對角線的元素值可能不為 0 外，其餘元素值均為 0。
  - 亦即，矩陣元素 $v_{i,j} = 0$ ， $i \neq j$。

$$DM_{5x5} = \begin{bmatrix} 2 & 0 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 & 0 \\ 0 & 0 & 9 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 88 \end{bmatrix}$$

圖 2.6 對角線矩陣

2.3.5 對角線矩陣的表示法

- 對角線矩陣 (Diagonal Matrix)
  - 對角線矩陣 DM[m][m] 共有 m 個非 0 元素。
  - 我們可以直接用一個一維陣列 X[m] 來表示對角線矩陣。
  - 亦即 X[i]= DM[i][i] 。

2.3.6 下三角矩陣的表示法

- 下三角矩陣 (Lower Triangular Matrix)
  - 它是一個「方陣」。
  - 對角線以上 (不含對角線) 的元素值均為 0 。

$$\begin{bmatrix} X_{0,0} & 0 & 0 & \cdots & 0 & 0 \\ X_{1,0} & X_{1,1} & 0 & \cdots & 0 & 0 \\ X_{2,0} & X_{2,1} & X_{2,2} & \cdots & 0 & 0 \\ & & & \cdots & 0 & 0 \\ X_{n-2,0} & X_{n-2,1} & X_{n-2,2} & \cdots & X_{n-2,n-2} & 0 \\ X_{n-1,0} & X_{n-1,1} & X_{n-1,2} & \cdots & X_{n-1,n-2} & X_{n-1,n-1} \end{bmatrix}$$

圖 2.7 下三角矩陣

2.3.6 下三角矩陣的表示法

- 假設
  - 下三角矩陣式的陣列 X 在記憶體之起始位置為 α 。
  - 且每個陣列元素所需的記憶體空間為 z 。
- 採「以列為主」的儲存策略，則

$$Loc_{RM}(X_{i,j}) = \alpha + ((i \times (i+1))/2 + j) \times z \text{ 。}$$

2.3.6 下三角矩陣的表示法

【例 1 】設 X[n][n] 為一個下三角矩陣，若要用一個一維陣列 Y[m] 來表示 X ，則 m 的最小值應該宣告為何？

【解答】

$$m = 1+2+3+\ldots+n$$
$$= n(n+1)/2 。$$

2.3.7 上三角矩陣的表示法

- 上三角矩陣 (Upper Triangular Matrix)
  - 它是一個「方陣」。
  - 對角線以下（不含對角線）的元素值均為 0。

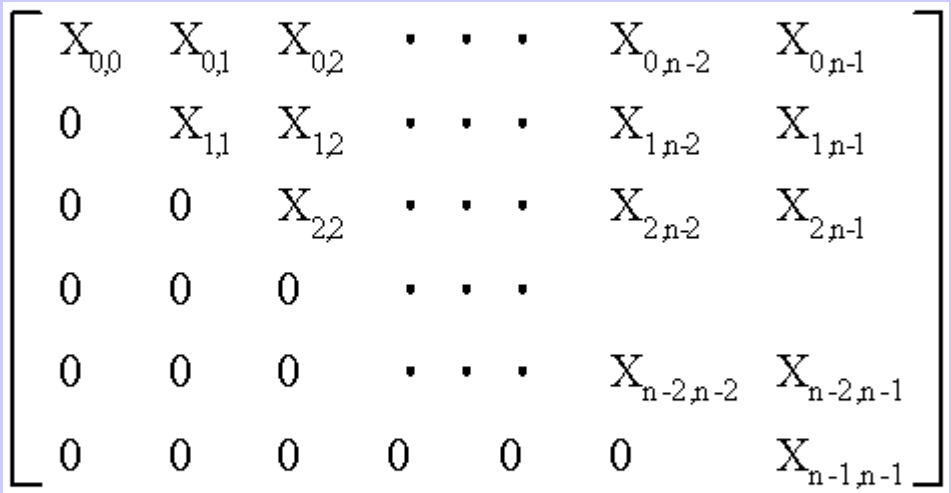


圖 2.8 上三角矩陣

2.3.7 上三角矩陣的表示法

- 假設
  - 下三角矩陣式的陣列 X 在記憶體之起始位置為 α 。
  - 且每個陣列元素所需的記憶體空間為 z 。
- 採「以行為主」的儲存策略，則

$$Loc_{CM}(X_{i,j}) = \alpha + ((j \times (j+1))/2 + i) \times z \text{ 。}$$

2.4 一維陣列的應用

- 2.4.1 計數器 (Counter)
- 2.4.2 暫存 (Temporary Store)
- 2.4.3 取代 (Substitution)
- 2.4.4 索引 (Indexing)
- 2.4.5 搜尋 (Searching)
- 2.4.6 排序 (Sorting)

2.4.1 計數器 (Counter)

- i = i + 1; (或 i++;) 。

- int count[n]; // 宣告

  ...

  count[i] = count[i] + 1; // 計數器 + 1

  或

  count[i]++ ; // 計數器 + 1

2.4.2 暫存 (Temporary Store)

- 「質數」：大於 1 的數，除了 1 與本身以外沒有任何一個數能夠整除它時，該數稱為質數。
  - 2 是最小的質數。
  - 3、 5、 7 均是質數。
  - 4、 6 不是質數，因為 4 可以被 2 整除， 6 可以被 2、 3 整除 。

2.4.2 暫存 (Temporary Store)

- 如何找出小於等於 100 的所有質數呢？
- 第一種方法：
  - 根據質數的定義，判斷 5 是否為質數時，須將 5 除以 2、 3、 4 ，只要能被其中一個數整除，那麼 5 就非質數，否則 5 就是質數。
  - 依此類推，判斷 x 是否為質數時，須將 x 除以 2、 3、 4...、 x-1 ，只要能被其中一個數整除，那麼 x 就非質數，否則 x 就是質數。

2.4.2 暫存 (Temporary Store)

- 第二種方法：
  - 將第 1 個找到的質數暫存於 prime[0] ，將第 2 個找到的質數暫存於 prime[1] ，將第 3 個找到的質數暫存於 prime[2] ，依此類推。
  - 要判斷一個整數 x 是否為質數時，我們只須取出 prime 陣列裡已經找到的質數來當做除數，將 x 除以這些質數，如果都不能被這些質數整除，即可斷定 x 為質數。
  - 因此，要判斷 6 是否為質數時，只須分別判斷 6 是否能被 2、 3、 5 整除即可，只要能被其中一個質數整除， 6 就非質數。因 6 能被 2 整除，非質數，不須再判斷能否被 3、 5 整除了。

2.4.3 取代 (Substitution)

- 要印出

```
♠A ♠2 ♠3 ♠4 ♠5 ♠6 ♠7 ♠8 ♠9 ♠10 ♠J ♠Q ♠K
♣A ♣2 ♣3 ♣4 ♣5 ♣6 ♣7 ♣8 ♣9 ♣10 ♣J ♣Q ♣K
♦A ♦2 ♦3 ♦4 ♦5 ♦6 ♦7 ♦8 ♦9 ♦10 ♦J ♦Q ♦K
♥A ♥2 ♥3 ♥4 ♥5 ♥6 ♥7 ♥8 ♥9 ♥10 ♥J ♥Q ♥K
```

- 則可用陣列來儲存花色和點數以取代之

```
15 char cf[4] = {6, 5, 4, 3}; /* 黑桃,梅花,方塊,紅心 */
16 char cp[13][2] = {"A ", "2 ", "3 ", "4 ", "5 ", "6 ", "7 ",
17         "8 ", "9 ", "10", "J ", "Q ", "K "};
```

2.4.4 索引 (Indexing)

- 「索引」：利用某一個陣列的元素值來當做另一個陣列的索引值。
- 尋找出眾數及其出現頻率
  - 一維陣列 x[15] 是一個含有 15 個元素的整數陣列，且陣列中的元素值均介於 0 與 5 之間。我們稱陣列 x 中出現次數最多的數為「眾數」。
  - 宣告一個整數陣列 count[6] ，然後用 count[i] 來記錄 x 陣列中 i 的出現次數。
  - 亦即 count[x[i]] = count[x[i]]++ ;

2.4.5 搜尋 (Searching)

- 搜尋 (Search) ：從一群資料中找出滿足特定條件的資料之動作稱之。
- 搜尋條件通常有：
  - 大於、等於、小於、大於或等於、不等於、小於或等於等多種組合。

2.4.5.1 循序搜尋法 (Sequential Search)

- 循序搜尋法 (Sequential Search) ：就是從頭到尾一筆一筆地搜尋、比較。

d[0]	d[1]	d[2]	d[3]	d[4]	d[5]	d[6]	d[7]
48	14	121	50	87	120	33	93

圖 2.9 一維整數陣列 d[8]的元素值

【例 1 】找出鍵值大於 70 的資料？

- 必須從 d[0] 開始逐筆加以比對，直到比完 d[7] 為止 (共計比較 8 次)
- 結果找到 d[2]、 d[4]、 d[5]、 d[7] 等 4 筆資料

d[0]	d[1]	d[2]	d[3]	d[4]	d[5]	d[6]	d[7]
48	14	121	50	87	120	33	93

圖 2.10 一維整數陣列 d[8]中，鍵值大於 70 的資料

2.4.5.2 二元搜尋法 (Binary Search)

- 二元搜尋法的特徵如下：
  - 資料須事先經過排序。
  - 每次都從可能有符合條件的一組資料中，拿第中間筆資料 (假設 d[i]) 出來跟鍵值 k 相比。
  - 每次比較完之後，大約可以捨去一半的資料，所以搜尋的速度很快。

2.4.5.2 二元搜尋法 (Binary Search)

【例 2 】二元搜尋法。

- 找出鍵值等於 120 的資料？

d[0]	d[1]	d[2]	d[3]	d[4]	d[5]	d[6]	d[7]
14	33	48	50	87	93	120	121

圖 2.11 一維整數陣列 d[8]的元素值(已按從小到大的順序排序)

	d[0]	d[1]	d[2]	d[3]	d[4]	d[5]	d[6]	d[7]
回合	14	33	48	50	87	93	120	121
1				x				
2						x		
3							x	
結果							*	

表 2.1 「找出鍵值等於 120 的資料」之每一回合

陰影：表示該回合的搜尋區間，x：表示該回合的第中間筆資料，

*：表示滿足搜尋條件的元素

2.4.5.2 二元搜尋法 (Binary Search)

【例 2 】二元搜尋法。

■找出鍵值大於 70 的資料？

d[0]	d[1]	d[2]	d[3]	d[4]	d[5]	d[6]	d[7]
14	33	48	50	87	93	120	121

圖 2.11 一維整數陣列 d[8]的元素值(已按從小到大的順序排序)

	d[0]	d[1]	d[2]	d[3]	d[4]	d[5]	d[6]	d[7]
回合	14	33	48	50	87	93	120	121
1				x				
2						x		
3					x			
結果					*	*	*	*

表 2.2 「找出鍵值大於 70 的資料」之每一回合

陰影：表示該回合的搜尋區間，x：表示該回合的第中間筆資料，

*：表示滿足搜尋條件的元素

2.4.6 排序 (Sorting)

- 排序 (SORT) ：乃將原始資料依照某個特定的次序加以重新排列。
- 排序時須指定：
  - 用哪一個資料項來排序？

    被用來排序的資料項又稱為「鍵 (Key) 」
  - 排序的順序
    - ✓遞增：即由小到大。
    - ✓遞減：即由大到小。

2.4.6.1 氣泡浮昇排序法 (Bubble Sort)

- 假設我們已經宣告了一個整數陣列 d[n] ，用來存放編號為 0 、 1 、 2 、 ... 、 n-1 的 n 筆資料。且

  d[0] = v_0、

  d[1] = v_1、

  d[2] = v_2、 ...

  d[n-2] = v_{n-2}、

  d[n-1] = v_{n-1}。

2.4.6.1 氣泡浮昇排序法 (Bubble Sort)

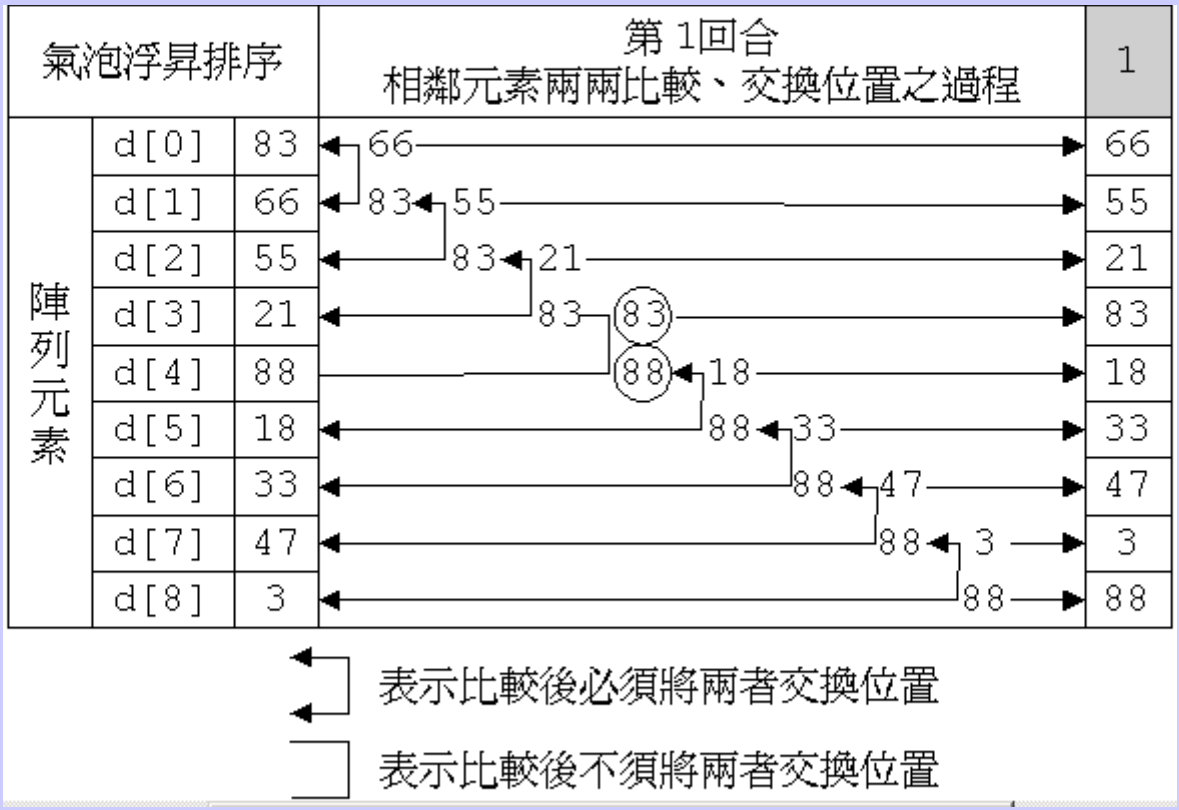
- 採用氣泡浮昇排序法的將一組資料依「鍵值遞增」排序之步驟說明如下：
  - 第 1 回合：
    - ✓從第 0 筆資料開始一直到第 n-1 筆資料，持續比較相鄰兩筆資料之大小，若 v_i 大於 v_{i+1} ，則須將兩者交換位置，亦即須將 v_i 儲存到 d[i+1] ， v_{i+1} 儲存到 d[i] 。換言之，必須保持小的在前，大的在後之順序。
    - ✓在第 1 回合處理完後，陣列中最大的數值已經被存放到 d[n-1] 的位置。

2.4.6.1 氣泡浮昇排序法 (Bubble Sort)

- 第 2 回合：
  - ✓從第 0 筆資料開始一直到第 n-2 筆資料，持續比較相鄰兩筆資料之大小，若 v_i 大於 v_{i+1} ，則須將兩者交換位置，亦即須將 v_i 儲存到 d[i+1] ， v_{i+1} 儲存到 d[i] 。
  - ✓在第 2 回合處理完後，陣列中第 2 大數的值已經被存放到 d[n-2] 的位置。
- 重複執行上述步驟，共計 (n-1) 回合，即可將陣列資料排列成由小到大的遞增順序。

2.4.6.1 氣泡浮昇排序法 (Bubble Sort)

【例 2 】氣泡浮昇排序法。



2.4.6.1 氣泡浮昇排序法 (Bubble Sort)

【例 2 】氣泡浮昇排序法。

表 2.3 氣泡浮昇排序法按「鍵值遞增」排序過程中每一回合的變化情形

氣泡浮昇排序法			每一回合排序處理後之結果							
			1	2	3	4	5	6	7	8
陣列資料	d[0]	83	66	55	21	21	18	18	18	3
	d[1]	66	55	21	55	18	21	21	3	18
	d[2]	55	21	66	18	33	33	3	21	21
	d[3]	21	83	18	33	47	3	33	33	33
	d[4]	88	18	33	47	3	47	47	47	47
	d[5]	18	33	47	3	55	55	55	55	55
	d[6]	33	47	3	66	66	66	66	66	66
	d[7]	47	3	83	83	83	83	83	83	83
	d[8]	3	88	88	88	88	88	88	88	88

2.5 二維陣列的應用

- 2.5.1 轉置矩陣 (Transpose Matrix)
- 2.5.2 對稱矩陣 (Symmetric Matrix)
- 2.5.3 反射矩陣 (Reflection Matrix)
- 2.5.4 矩陣相加 (Matrix Addition)
- 2.5.5 矩陣相減 (Matrix Subtraction)
- 2.5.6 矩陣乘積 (Matrix Multiplication)
- 2.5.7 稀疏矩陣 (Sparse Matrix)

2.5.1 轉置矩陣 (Transpose Matrix)

- 設 M 是一個 m×n 矩陣，而 TM 為其轉置矩陣，則：
  - TM 為一個 n×m 矩陣。
  - M 的第 0 列元素成為 TM 的第 0 行元素，
    M 的第 1 列元素成為 TM 的第 1 行元素，…，

    M 的第 m-1 列元素成為 TM 的第 m-1 行元素。
  - 亦即 TM[j][i] = M[i][j] 。

2.5.1 轉置矩陣 (Transpose Matrix)

【例 1 】轉置矩陣。

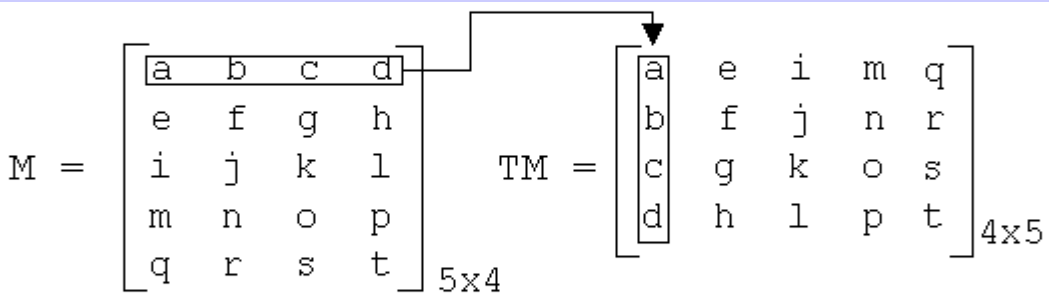


圖 2.13 原始矩陣 M_{5x4} 及其轉置矩陣 TM_{4x5}

2.5.2 對稱矩陣 (Symmetric Matrix)

- 對稱矩陣為一方陣，且元素值 $v_{ij} = v_{ji}$。

$$SM_{3x3} = \begin{bmatrix} 2 & 1 & 3 \\ 1 & 3 & 0 \\ 3 & 0 & 9 \end{bmatrix}$$

圖 2.14 對稱矩陣 SM_{3x3}

2.5.3 反射矩陣 (Reflection Matrix)

- 反射矩陣 (RM_{mxn}) 是將原始矩陣 (M_{mxn}) 進行水平方向的鏡射。

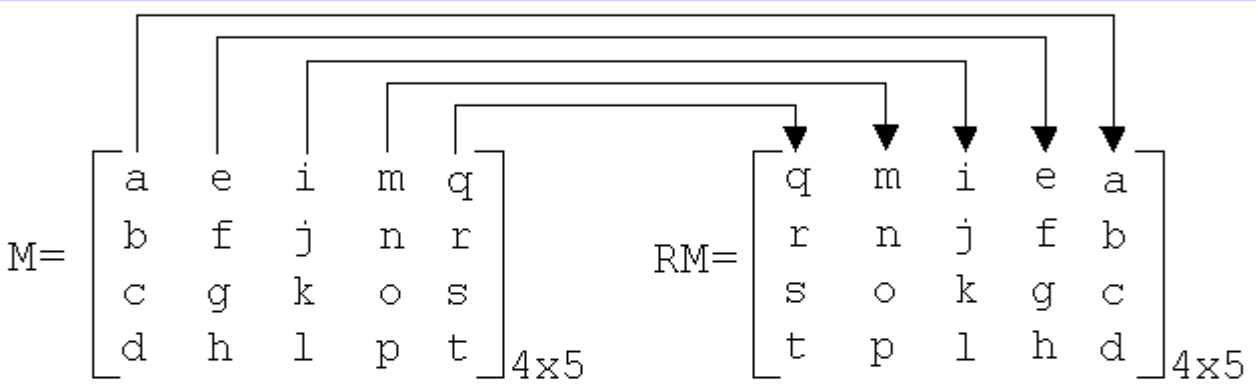


圖 2.15 原始矩陣 M 與其反射矩陣 RM 的關係

2.5.4 矩陣相加 (Matrix Addition)

- $c_{ij}=a_{ij}+b_{ij}$。

【例 1 】矩陣相加。

$$A_{2x3}=\begin{bmatrix}1 & 0 & 1\\ 2 & 3 & 9\end{bmatrix} \qquad B_{2x3}=\begin{bmatrix}2 & 2 & 1\\ 1 & 3 & 0\end{bmatrix}$$

$$C_{2x3}=A_{2x3}+B_{2x3}=\begin{bmatrix}3 & 2 & 2\\ 3 & 6 & 9\end{bmatrix}$$

2.5.5 矩陣相減 (Matrix Subtraction)

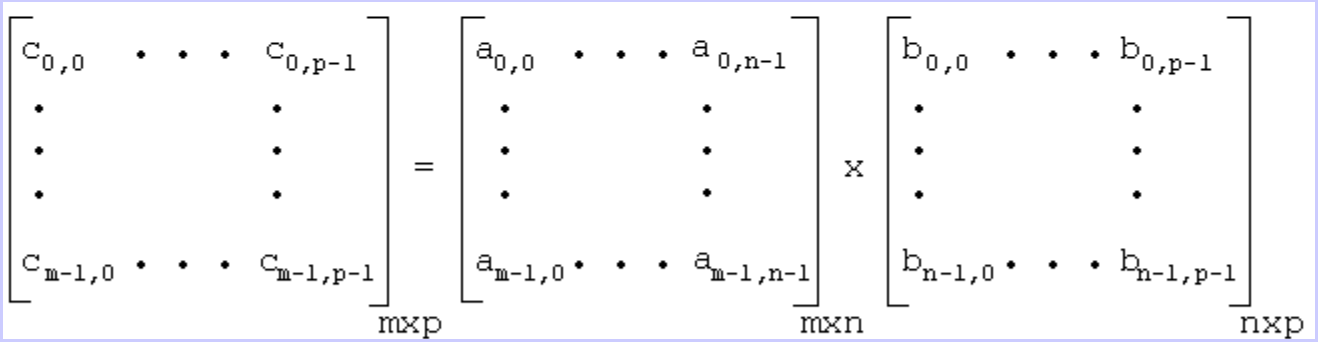
- $c_{ij}=a_{ij}-b_{ij}$。

【例１】矩陣相減。

$$A_{2x3}=\begin{bmatrix}1 & 0 & 1\\ 2 & 3 & 9\end{bmatrix} \quad B_{2x3}=\begin{bmatrix}2 & 2 & 1\\ 1 & 3 & 0\end{bmatrix}$$

$$C_{2x3}=A_{2x3}-B_{2x3}=\begin{bmatrix}-1 & -2 & 0\\ 1 & 0 & 9\end{bmatrix}$$

2.5.6 矩陣乘積 (Matrix Multiplication)



$$c_{0,0} = a_{0,0} \times b_{0,0} + a_{0,1} \times b_{1,0} + \ldots + a_{0,n-1} \times b_{n-1,0}$$

$$c_{0,1} = a_{0,0} \times b_{0,1} + a_{0,1} \times b_{1,1} + \ldots + a_{0,n-1} \times b_{n-1,1}$$

$$\ldots$$

$$c_{0,p-1} = a_{0,0} \times b_{0,p-1} + a_{0,1} \times b_{1,p-1} + \ldots + a_{0,n-1} \times b_{n-1,p-1}$$

$$\ldots$$

$$c_{m-1,0} = a_{m-1,0} \times b_{0,0} + a_{m-1,1} \times b_{1,0} + \ldots + a_{m-1,n-1} \times b_{n-1,0}$$

$$\ldots$$

$$c_{m-1,p-1} = a_{m-1,0} \times b_{0,p-1} + a_{m-1,1} \times b_{1,p-1} + \ldots + a_{m-1,n-1} \times b_{n-1,p-1}$$

2.5.6 矩陣乘積 (Matrix Multiplication)

【例 1 】矩陣乘積。

設 $A_{2x2} = \begin{bmatrix} 2 & 1 \\ 4 & 3 \end{bmatrix}$ ， $B_{2x3} = \begin{bmatrix} 3 & 2 & 1 \\ 6 & 5 & 4 \end{bmatrix}$ ，則

$$C_{2x3} = A_{2x2} \times B_{2x3} = \begin{bmatrix} 2 & 1 \\ 4 & 3 \end{bmatrix} \times \begin{bmatrix} 3 & 2 & 1 \\ 6 & 5 & 4 \end{bmatrix} = \begin{bmatrix} 12 & 9 & 6 \\ 30 & 23 & 16 \end{bmatrix}$$

2.5.7 稀疏矩陣 (Sparse Matrix)

- 一個矩陣中，若大多數的元素值均為 0 ，則稱該矩陣為「稀疏矩陣」。
- 這裡所謂的元素值為 0 在應用上包括幾種涵義：
  - 真的為 0 。
  - 為虛值 (Null Value)，亦即沒有資料值

$SM_{6x7}=$

	0	1	2	3	4	5	6
0	1	0	0	0	0	0	0
1	0	0	0	0	0	91	0
2	0	0	0	0	0	0	0
3	0	88	0	0	0	0	0
4	0	0	0	0	0	0	0
5	0	0	2	0	0	3	0

圖 2.16 一個典型的稀疏矩陣 SM_{6x7}

2.5.7 稀疏矩陣 (Sparse Matrix)

【例 1 】稀疏矩陣的表示法。

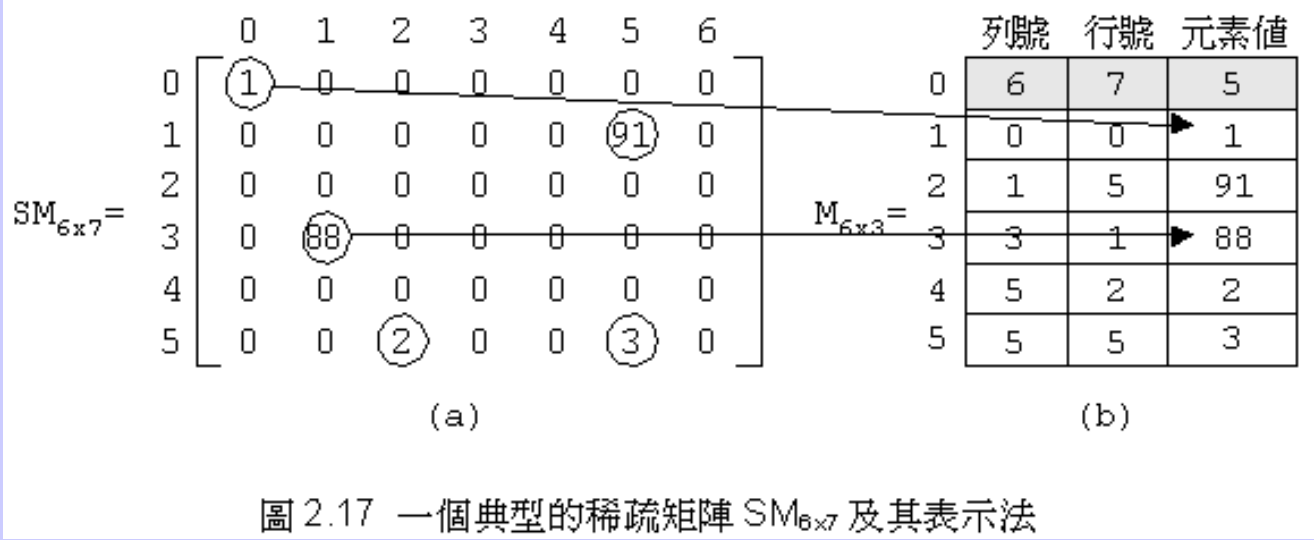


圖 2.17 一個典型的稀疏矩陣 SM_{6x7} 及其表示法

2.5.7 稀疏矩陣 (Sparse Matrix)

【例 2 】試設計一個用以表示三維稀疏矩陣的表示法。

【解答】設三維稀疏矩陣 SMmxnxo 各維之大小分別為 m、 n、 o ，且稀疏矩陣中非 0 元素

共計 個

$M_{(p+1)\times 4}$ =

	第1維	第2維	第3維	元素值
0	m	n	o	p
1				
2				
3				
p				

圖 2.18 三維稀疏矩陣 SM_{mxnxo} 的二維陣列表示法

練習程式

- 兩個稀疏矩陣相加及相乘，計算結果用傳統矩陣的方式呈現

5	5	3
0	2	1
3	4	2
4	0	3

5	5	2
3	4	4
4	3	5